



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
Disciplina: ALGORITMO E ESTRUTURA DE DADOS I

Semestre: 2026.2

Data: ____/____/____
PROFESSORA: Rosana Rego

— Exercício 3: Unidade I —

Union e Enum: *tagged unions* em structs

Nome: _____

Nota: _____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade3.c`, evolução da `atividade2.c`, cujo objetivo é demonstrar, de forma visual e interativa (usando a biblioteca **raylib**), os seguintes conceitos de linguagem C:

- Definição de uma **union** (`DadosItem`) cujos campos compartilham o mesmo espaço de memória;
- Uso de um **enum** (`TipoItem`) como “tag” para saber qual campo da *union* é válido em cada momento (padrão *tagged union*);
- Uma *struct* (`Item`) combinando **enum** + **union** + campos comuns;
- **Alocação dinâmica** de um vetor de *structs* com `malloc` e liberação com `free`;
- **Ponteiros para struct** (`Jogador *`, `Item *`) passados para funções que alteram o dado original.

O programa exibe um jogador que coleta itens espalhados pela tela: alguns são armas (aumentam o dano) e outros são poções (curam vida).

1 Duas structs auxiliares e a union

Uma arma e uma poção guardam dados completamente diferentes. Como cada `Item` é **ou** uma arma **ou** uma poção (nunca os dois ao mesmo tempo), esses dados podem compartilhar o mesmo espaço de memória através de uma **union**:

```
1 typedef struct {
2     float dano;
3     int  alcance;
4 } DadosArma;
5
6 typedef struct {
7     int cura;
8 } DadosPocao;
9
10 typedef union {
11     DadosArma  arma;
12     DadosPocao pocao;
13 } DadosItem;
```

Diferentemente de uma *struct* (em que cada campo ocupa seu próprio espaço), uma **union** reserva memória suficiente apenas para o **maior** dos seus campos, e todos os campos começam no mesmo endereço. Por isso, escrever em `dados.arma` e depois ler `dados.pocao` produziria lixo – é preciso sempre saber, por fora, qual campo é o válido.

2 O enum como tag da union

É exatamente esse o papel do campo `tipo`: ele diz, para cada *Item*, qual campo da *DadosItem* deve ser lido.

```
1 typedef enum {
2     ITEM_ARMA ,
3     ITEM_POCAO
4 } TipoItem;
5
6 typedef struct {
7     Vector2    pos;
8     float      raio;
9     TipoItem   tipo;
10    DadosItem  dados;
11    bool        coletado;
12 } Item;
```

3 Preenchendo a union de acordo com o tipo sorteado

Ao criar o vetor dinâmico de itens, o tipo é sorteado primeiro; só então o campo correspondente da *union* é preenchido:

```
1 Item *criarItens(int quantidade) {
2     Item *itens = (Item *)malloc(quantidade * sizeof(Item));
3     for (int i = 0; i < quantidade; i++) {
4         Item *it = (itens + i);
5         it->pos      = (Vector2){ GetRandomValue(30, LARGURA_JANELA -
6                                     30),
7                                     GetRandomValue(30, ALTURA_JANELA -
8                                     30) };
9         it->raio      = 12.0f;
10        it->coletado   = false;
11        it->tipo      = (TipoItem)GetRandomValue(ITEM_ARMA, ITEM_POCAO
12                                                    );
13
14        if (it->tipo == ITEM_ARMA) {
15            it->dados.arma.dano      = (float)GetRandomValue(2, 8);
16            it->dados.arma.alcance   = GetRandomValue(1, 3);
17        } else {
18            it->dados.pocao.cura     = GetRandomValue(10, 30);
19        }
20    }
21    return itens;
22 }
```

4 Aplicando o efeito via ponteiro para struct

A função `aplicarItem` recebe **ponteiros para struct** do jogador e do item, e usa o campo `tipo` (enum) para decidir qual campo da *union* deve ser lido:

```

1 void aplicarItem(Jogador *j, Item *item) {
2     switch (item->tipo) {
3         case ITEM_ARMA:
4             j->dano += item->dados.arma.dano;
5             break;
6         case ITEM_POCAO:
7             j->vida += item->dados.pocao.cura;
8             break;
9     }
10    item->coletado = true;
11 }

```

Ciclo de vida da memória

Etapa	Vetor de itens (Item *)
Alocação	criarItens()
Uso	aplicarItem() + desenharItem()
Liberação	free(itens)

Assim como matriz e vetor de *structs* da atividade 1, o vetor de itens é um único bloco contíguo alocado com `malloc` e deve ser liberado com `free` antes do fim do programa.

5 Exercícios

Exercício 1 — Novo tipo de item: escudo

Atualmente a *union* `DadosItem` só conhece dados de arma e de poção.

Tarefa: adicione um terceiro tipo de item, `ITEM_ESCUDO`, que aumenta uma nova propriedade: armadura do jogador.

Dicas:

- Crie uma *struct* `DadosEscudo` com um campo `int` `absorcao`.
- Adicione esse tipo como um novo campo da *union* `DadosItem`.
- Acrescente `ITEM_ESCUDO` ao `enum` `TipoItem` e trate o novo caso em `criarItens`, `aplicarItem` e `desenharItem`.
- Lembre-se de adicionar o campo `int` `armadura` à *struct* `Jogador`.

Exercício 2 — Poção envenenada

Nem toda poção precisa curar: algumas podem ser armadilhas.

Tarefa: com uma certa probabilidade, uma poção deve ter *cura* negativa (causa dano em vez de curar), sem alterar o **enum** `TipoItem`.

Dicas:

- Em `criarItens`, sorteie um número extra (0 a 9); se for menor que 3, defina `it->dados.pocao.cura` como um valor negativo.
- Em `aplicarItem`, garanta que `j->vida` nunca fique negativo após aplicar o efeito (use um `if` para travar em zero).
- Desenhe poções envenenadas com uma cor diferente (por exemplo, roxo) para dar uma pista visual ao jogador – isso exige guardar mais uma informação na *struct* `Item` ou recalcular a cor a partir do sinal de *cura*.